⚡ RIMS — Rental Item Management System

> **High-Precision Enterprise Equipment & Asset Leasing Platform**
> Developed with **Java 21**, **Spring Boot 3**, **Spring Data JPA**, **Hibernate ORM**, **SQL Database (H2 / PostgreSQL / MySQL)**.

📋 TABLE OF CONTENTS

1. Platform Overview
2. System Architecture
3. End-to-End Workflow & Flow Diagrams
4. Relational Data Model & ER Diagram
5. Complete Feature Matrix by Stakeholder
   - Public & Discovery Features
   - Customer / Buyer Portal
   - Institutional Seller Hub
   - SUDO Admin Command Terminal
6. Pricing, Duration & Escrow Deposit Calculation
7. Security & Authentication Architecture
8. Razorpay Payment Gateway Integration
9. Pre-Seeded Demo Accounts
10. Database Configuration & Persistence
11. Installation & Execution Guide

🌟 PLATFORM OVERVIEW

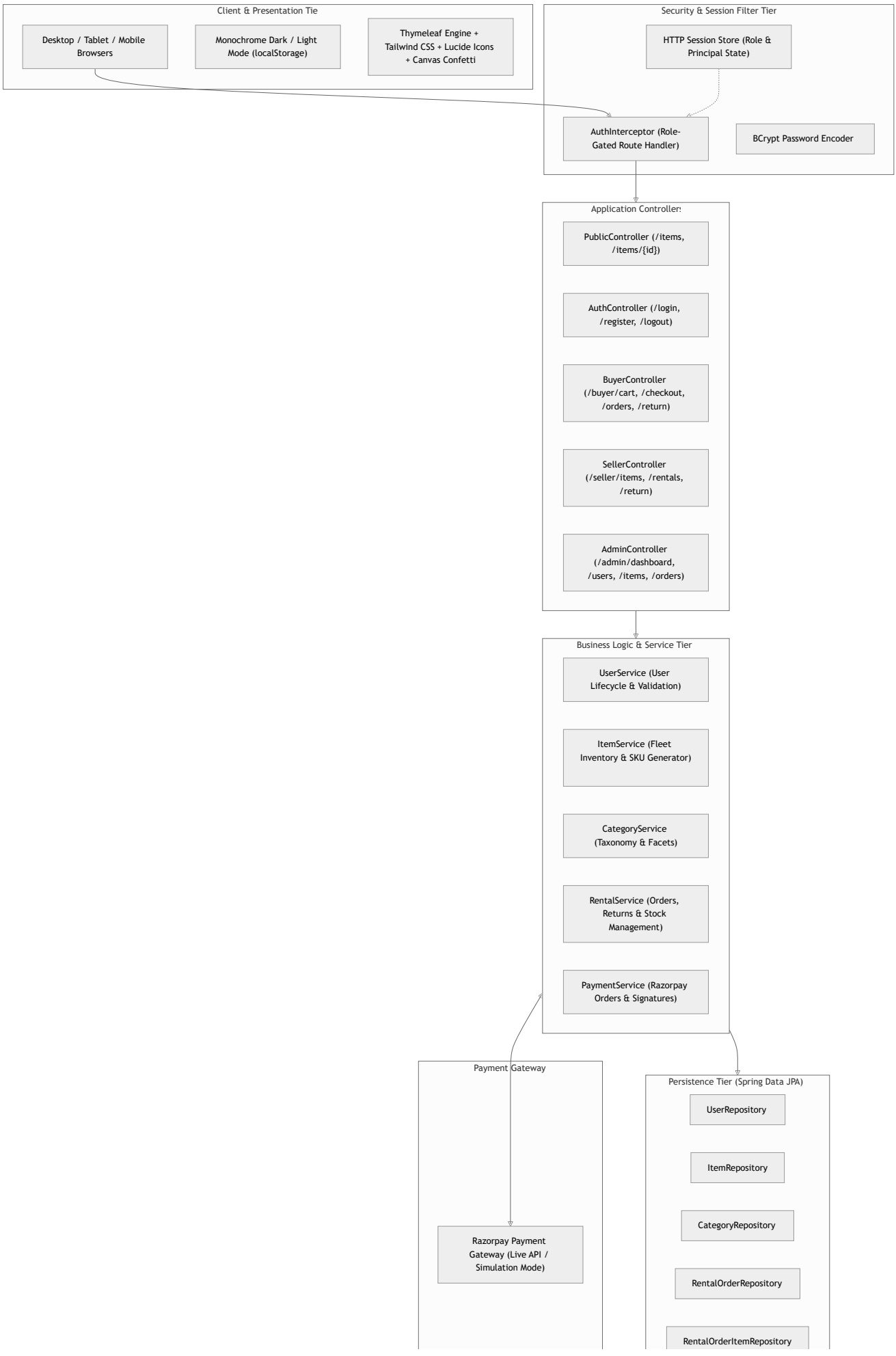
RIMS (Rental Item Management System) is an institutional-grade equipment reservation and asset leasing platform connecting verified equipment providers (studios, heavy industrial fleets, audio staging providers, and surveying laboratories) with commercial clients.

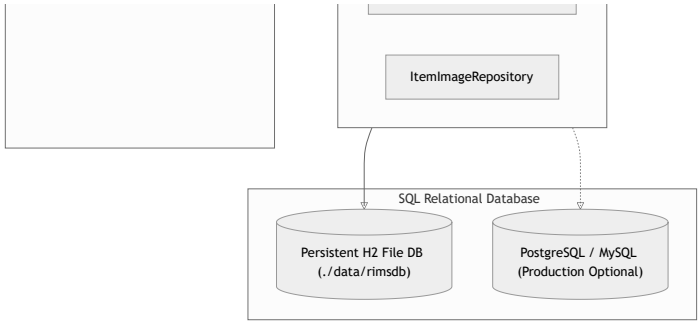
The system is designed with:

- **Absolute Transparency:** Full visibility into daily rates, duration scaling, and escrow security deposits.
- **Serial Traceability:** Guaranteed unique product SKU codes (e.g. `RIMS-CAM-8001`, `RIMS-LGT-2044`) preserved as historical order snapshots.
- **Closed-Loop Lifecycle:** Complete tracking from reservation, Razorpay settlement, equipment deployment, client return initiation, seller check-in verification, automated inventory stock replenishment, to escrow deposit release.
- **Monochrome Luxury Design:** High-contrast black-and-white minimalist interface with glassmorphism navigation, responsive layouts, and real-time Dark/Light theme toggle.

🏗️ SYSTEM ARCHITECTURE

RIMS is built on a clean multi-tiered enterprise architecture utilizing Spring Boot 3, Spring Data JPA, and server-side Thymeleaf rendering styled with Tailwind CSS.

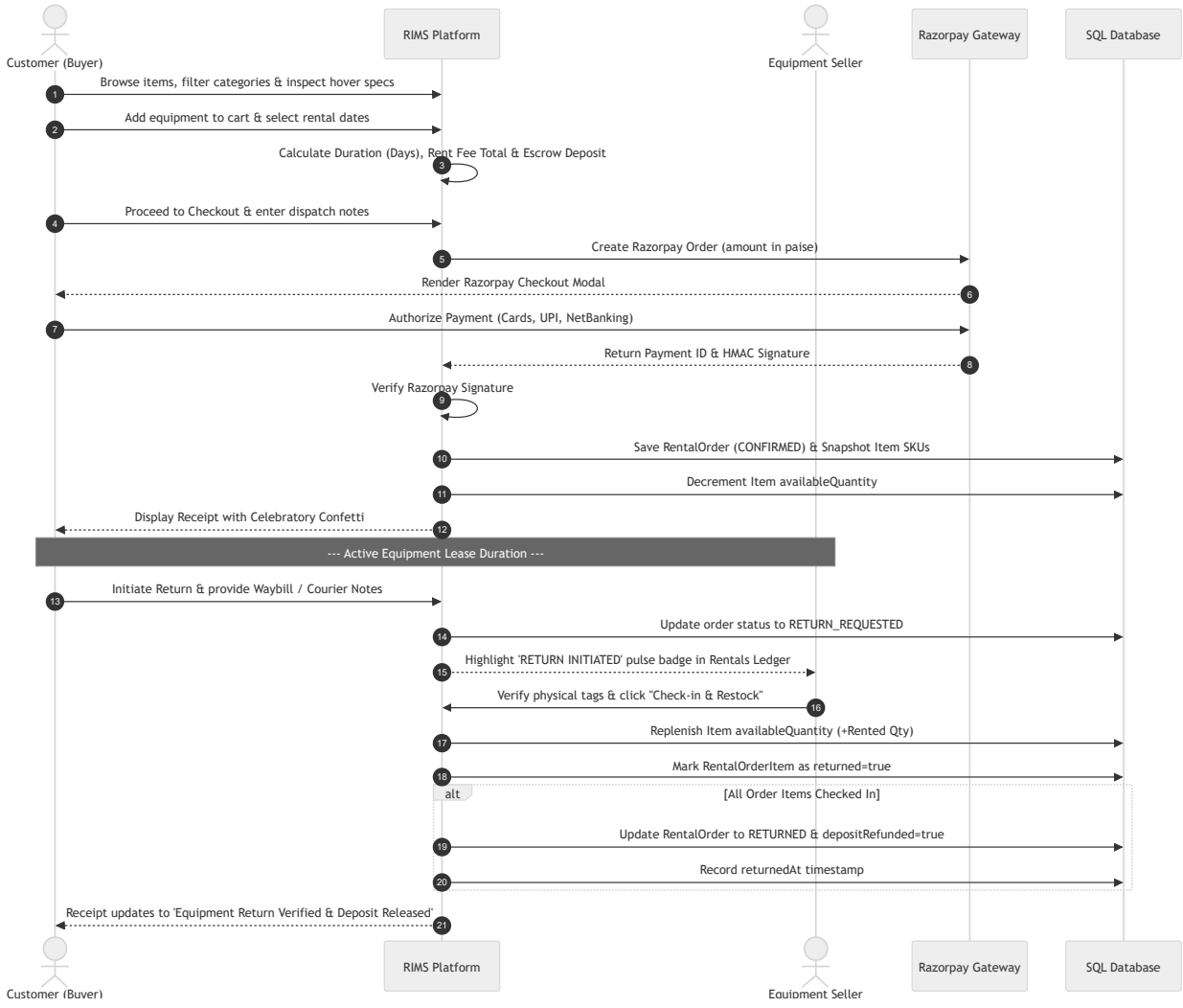

Client & Presentation Tie
Desktop / Tablet / Mobile Browsers
Monochrome Dark / Light Mode (localStorage)
Thymeleaf Engine + Tailwind CSS + Lucide Icons + Canvas Confetti
Security & Session Filter Tier
HTTP Session Store (Role & Principal State)
AuthInterceptor (Role-Gated Route Handler)
BCrypt Password Encoder
Application Controllers
PublicController (/items, /items/{id})
AuthController (/login, /register, /logout)
BuyerController (/buyer/cart, /checkout, /orders, /return)
SellerController (/seller/items, /rentals, /return)
AdminController (/admin/dashboard, /users, /items, /orders)
Business Logic & Service Tier
UserService (User Lifecycle & Validation)
ItemService (Fleet Inventory & SKU Generator)
CategoryService (Taxonomy & Facets)
RentalService (Orders, Returns & Stock Management)
PaymentService (Razorpay Orders & Signatures)
Payment Gateway
Razorpay Payment Gateway (Live API / Simulation Mode)
Persistence Tier (Spring Data JPA)
UserRepository
ItemRepository
CategoryRepository
RentalOrderRepository
RentalOrderItemRepository



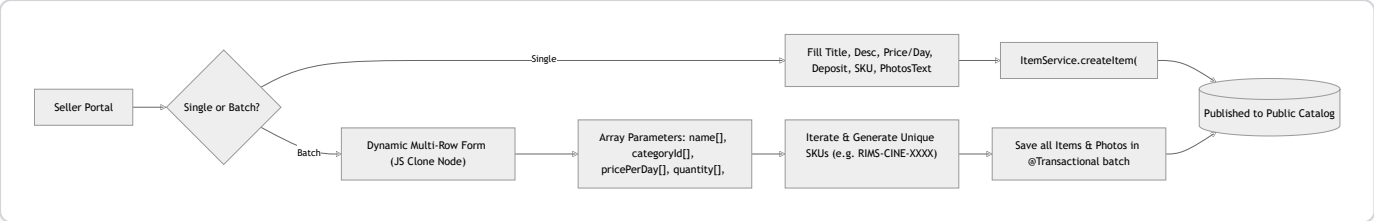
🔄 END-TO-END WORKFLOW & FLOW DIAGRAMS

1. Complete Rental & Return Lifecycle

The diagram below illustrates the end-to-end journey from product catalog discovery to post-rental inventory restocking:

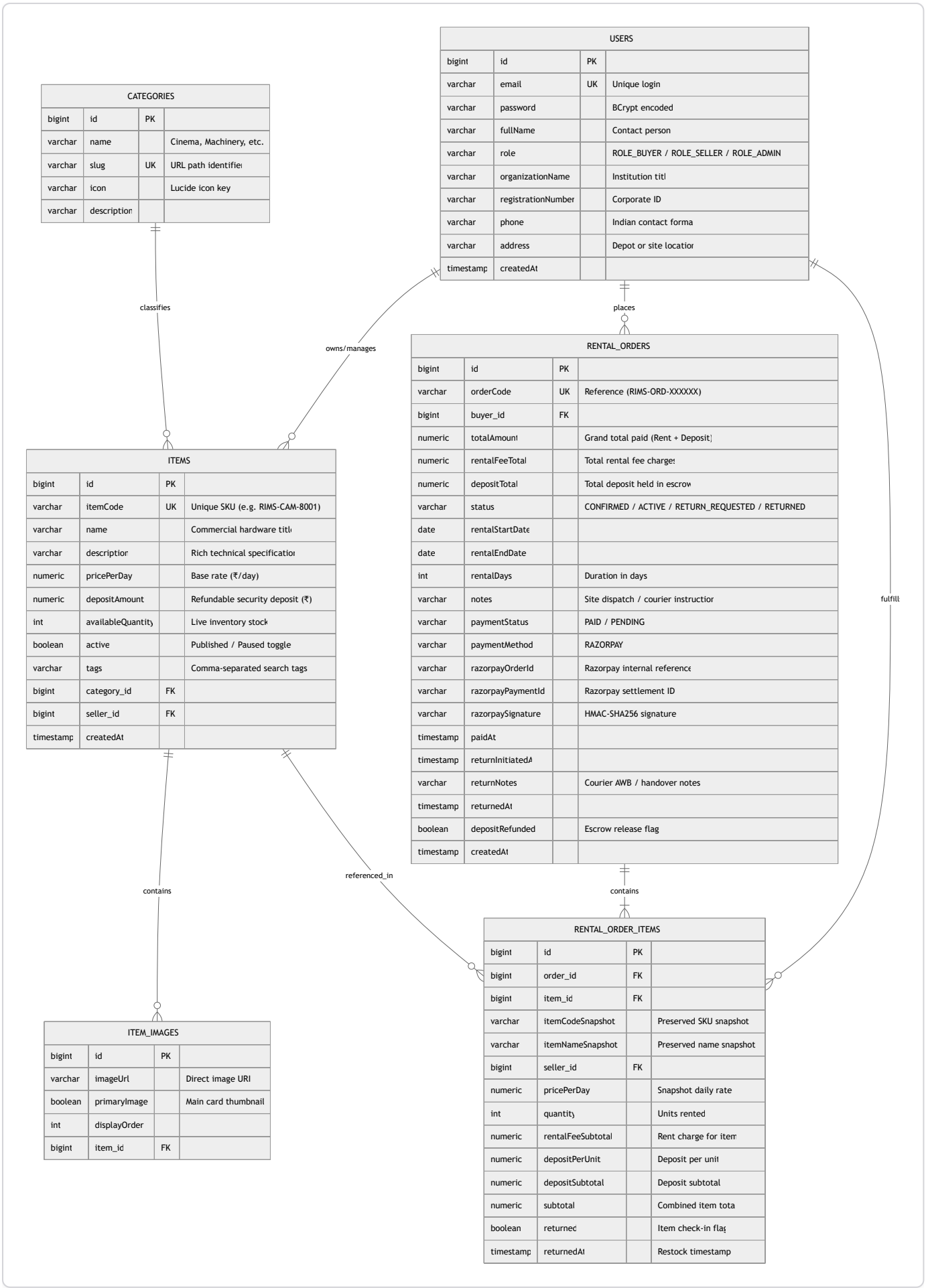


2. Seller Batch Equipment Onboarding Flow



📊 RELATIONAL DATA MODEL & ER DIAGRAM

The database utilizes foreign keys and historical snapshot fields so that even if a seller edits an item or an admin purges a listed product, existing client order records and financial audit trails remain completely immutable.


USERS
bigint id PK
varchar email UK Unique login
varchar password BCrypt encoded
varchar fullName Contact person
varchar role ROLE_BUYER / ROLE_SELLER / ROLE_ADMIN
varchar organizationName Institution titl
varchar registrationNumber Corporate ID
varchar phone Indian contact forma
varchar address Depot or site locatior
timestamp createdAt
CATEGORIES
bigint id PK
varchar name Cinema, Machinery, etc.
varchar slug UK URL path identifie
varchar icon Lucide icon key
varchar description
classifies
owns/manages
places
fulfill
RENTAL_ORDERS
bigint id PK
varchar orderCode UK Reference (RIMS-ORD-XXXXXX)
bigint buyer_id FK
numeric totalAmount Grand total paid (Rent + Deposit)
numeric rentalFeeTotal Total rental fee charges
numeric depositTotal Total deposit held in escrow
varchar status CONFIRMED / ACTIVE / RETURN_REQUESTED / RETURNED
date rentalStartDate
date rentalEndDate
int rentalDays Duration in days
varchar notes Site dispatch / courier instructior
varchar paymentStatus PAID / PENDING
varchar paymentMethod RAZORPAY
varchar razorpayOrderId Razorpay internal reference
varchar razorpayPaymentId Razorpay settlement ID
varchar razorpaySignature HMAC-SHA256 signature
timestamp paidAt
timestamp returnInitiatedA
varchar returnNotes Courier AWB / handover notes
timestamp returnedAt
boolean depositRefunded Escrow release flag
timestamp createdAt
ITEMS
bigint id PK
varchar itemCode UK Unique SKU (e.g. RIMS-CAM-8001)
varchar name Commercial hardware titl
varchar description Rich technical specificatio
numeric pricePerDay Base rate (₹/day)
numeric depositAmount Refundable security deposit (₹)
int availableQuantity Live inventory stock
boolean active Published / Paused toggle
varchar tags Comma-separated search tags
bigint category_id FK
bigint seller_id FK
timestamp createdAt
contains
referenced_in
contains
RENTAL_ORDER_ITEMS
bigint id PK
bigint order_id FK
bigint item_id FK
varchar itemCodeSnapshot Preserved SKU snapshot
varchar itemNameSnapshot Preserved name snapshot
bigint seller_id FK
numeric pricePerDay Snapshot daily rate
int quantity Units rented
numeric rentalFeeSubtotal Rent charge for item
numeric depositPerUnit Deposit per unit
numeric depositSubtotal Deposit subtotal
numeric subtotal Combined item tota
boolean returned Item check-in flag
timestamp returnedAt Restock timestamp
ITEM_IMAGES
bigint id PK
varchar imageUrl Direct image URI
boolean primaryImage Main card thumbnail
int displayOrder
bigint item_id FK

💎 COMPLETE FEATURE MATRIX BY STAKEHOLDER

1. Public Catalog & Discovery Features

- **Landing Page (`/`):**
  - Modern hero layout with call-to-actions, dynamic stats counter, and category showcase cards.
  - Dedicated Portal Access Cards for Customer Portal, Institutional Seller Hub, and SUDO Admin.
- **Sticky Glassmorphism Header:**
  - Real-time Lucide icons, responsive navigation links, live search bar, and active customer cart counter.
  - **Theme Switcher:** Seamless Dark Mode / Light Mode toggle with zero page reload, persisted in browser `localStorage`.
- **Universal Search & Filtering (`/items`):**
  - **Keyword Search:** Real-time query matching across hardware titles, rich descriptions, and SKU codes.
  - **Category Navigation:** One-click filter across 6 pre-seeded equipment sectors.
  - **Tag Cloud:** Filter by specialized tags (e.g. `#8K`, `#Cinema`, `#RED`, `#Hydraulic`, `#Lighting`).
  - **Sort Options:** Price: Low to High, Price: High to Low, and Newest Arrivals.
- **Interactive Hover-Reveal Cards:**
  - Displays high-resolution primary photo with category badge and unique SKU tag.
  - Smooth CSS 3D perspective transition on hover revealing detailed specifications, seller institution name, and live stock indicator.
- **Hardware Detail Screen (`/items/{id}`):**
  - Multi-photo interactive gallery with thumbnail selector.
  - Verified institution profile card including physical depot address and direct phone contact.
  - Duration estimator displaying rental rates and security deposit breakdown.

---

2. Customer / Buyer Portal

- **Customer Authentication:** Dedicated login (`/login/buyer`) and registration (`/register/buyer`) capturing Indian mobile numbers and site addresses.
- **Smart Rental Cart (`/buyer/cart`):**
  - Add multiple items from different institutional sellers into a single session-managed cart.
  - Increase/decrease quantities dynamically with stock validation limits.
  - Interactive date pickers for **Rental Start Date** and **Return Date**.
  - Instant duration computation ensuring a minimum 1-day rental period.
- **Transparent Checkout (`/buyer/checkout`):**
  - Explicit financial breakdown showing **Rental Fees** vs. **Refundable Escrow Security Deposit**.
  - Site Dispatch / Delivery instructions field (e.g., *"Deliver to Studio 4B; contact production manager on site"*).
- **Payment Settlement:**
  - Integrated with Razorpay modal checkout and local development simulation fallback.
  - Generates cryptographically secure Razorpay Order IDs and validates transaction signatures.
- **Customer Dashboard (`/buyer/dashboard`):**
  - Overview counters: Total Active Leases, Completed Rentals, and Total Expenditure.
  - Interactive orders table with booking dates, SKU breakdown, payment badges, and receipt actions.
- **Printable Monochrome Order Receipt (`/buyer/orders/{id}`):**
  - Clean, invoice-grade typography with itemized SKUs, quantities, and settled payment IDs.
  - Triggered celebratory confetti on newly confirmed orders.
- **Client Equipment Return Handover:**

- Dedicated **"Equipment Return Handover"** card on the order receipt.
- Allows clients to submit courier waybills, AWB tracking numbers, or drop-off depot notes.
- Immediate visual badge transitions: `Active` $\rightarrow$ `RETURN IN PROGRESS` $\rightarrow$ `✓ Returned`.

---

3. Institutional Seller Hub

- **Seller Authentication:** Dedicated portal (`/login/seller`) and registration (`/register/seller`) capturing corporate organization name and registration numbers.
- **Executive Seller Dashboard (`/seller/dashboard`):**
  - Fleet metrics: Total Listed Equipment, Active Field Deployments, Total Physical Inventory, and Gross Revenue ($₹$).
  - Quick-access tables for pending returns and fleet management.
- **Single Equipment Registration (`/seller/items/create`):**
  - Upload title, technical specifications, category classification, and custom or auto-generated SKUs.
  - Configure daily rental rate ($₹$), refundable security deposit ($₹$), and available inventory units.
  - Paste multiple image URLs with automated primary photo selection.
  - Add custom search tags.
- **Batch Equipment Onboarding:**
  - Client-side dynamic multi-row form allowing bulk registration of multiple pieces of equipment in one submission.
  - Individual category assignments, prices, quantities, and photo sets per row.
- **Fleet Inventory Ledger (`/seller/items`):**
  - Comprehensive equipment table with image previews, asset codes, pricing, and live available units.
  - One-click **Publish / Pause** toggle to instantly hide or activate items in the public catalog without deleting them.
- **Client Rentals Deployment Ledger (`/seller/rentals`):**
  - Full audit trail answering: **Who rented which asset, for how long, and what were the fees?**
  - Customer name, verified contact number, rental date window, duration, units leased, net earnings, and deposit held in escrow.
- **Inventory Restock & Return Check-In:**
  - Real-time `RETURN INITIATED` animated badge when a customer requests return.
  - One-click **"Check-in & Restock (+Qty)"** button.
  - Automatically increments `Item.availableQuantity` by the returned quantity.
  - Updates item status and releases customer escrow deposit once all items are verified.

---

4. SUDO Admin Command Terminal

- **Master Admin Authentication:** Dedicated terminal entrypoint (`/login/admin`).
- **Root Command Center (`/admin/dashboard`):**
  - Platform-wide telemetry: Total Users, Total Inventory Fleet, Total Orders, and Gross Turnover.
- **Universal Database Inspection (`/admin/users`):**
  - Complete transparency view inspecting every single SQL user row.
  - Inspects: User ID, Full Name, Email, Role badge, Organization Title, Corporate Registration Number, Phone, Physical Address, and Stored Password Credential Hash.
- **Master Fleet Delisting Authority (`/admin/items`):**
  - Global inventory control table across all sellers.
  - One-click **Delist / Remove** button to permanently purge inappropriate or obsolete products.
- **Global Orders Master Ledger (`/admin/orders`):**
  - Complete visibility over all transactions, Razorpay IDs, site notes, and order receipts across the platform.

💰 PRICING, DURATION & ESCROW DEPOSIT CALCULATION

RIMS uses a dual-component billing model to protect both the lessor and the lessee:

```
\text{Rental Duration (Days)} = \max\Big(1,\, \text{EndDate} - \text{StartDate}\Big)
```

$$\text{Item Rental Fee} = \text{Daily Rate} \times \text{Quantity} \times \text{Rental Duration}$$

$$\text{Item Security Deposit} = \text{Deposit Per Unit} \times \text{Quantity}$$

$$\text{Grand Total Payable} = \sum \text{Item Rental Fees} + \sum \text{Item Security Deposits}$$

- **Rental Fee:** Retained as vendor revenue upon confirmation.
- **Security Deposit:** Held securely in escrow and automatically released upon physical return check-in by the seller (`depositRefunded = true`).

🔒 SECURITY & AUTHENTICATION ARCHITECTURE

1. **Role-Based Access Control (RBAC):**
   - Three distinct user authorities: `ROLE_BUYER` , `ROLE_SELLER` , `ROLE_ADMIN` .
   - Enforced via `AuthInterceptor` which validates active HTTP session state and role permissions before controller dispatch.
2. **Dedicated Portals & Clean Isolation:**
   - Separate login paths (`/login/buyer` , `/login/seller` , `/login/admin`) prevent cross-role confusion.
   - Unauthorized requests automatically redirect to the relevant login portal with flash alerts.
3. **Data Integrity & Immutability:**
   - Order items maintain `itemCodeSnapshot` and `itemNameSnapshot` so that historical receipts remain tamper-proof.
   - Seller check-in endpoints verify ownership (`roi.getSeller().getId().equals(seller.getId())`) to prevent unauthorized inventory tampering.

💳 RAZORPAY PAYMENT GATEWAY INTEGRATION

- **Configuration:** Managed via `src/main/resources/application.properties` and `RazorpayConfig.java` .
- **Live / Sandbox Keys:**

```
razorpay.key.id=${RAZORPAY_KEY_ID:rzp_test_TlRxOr7yFmZtQx}
razorpay.key.secret=${RAZORPAY_KEY_SECRET:q82Waq7OyEyW4ffMdy30ov6w}
razorpay.currency=INR
```

- **Simulated Checkout Support:** Includes a built-in simulation fallback (`/buyer/payment/simulate`) enabling rapid local testing without hitting live banking networks.

👥 PRE-SEEDED DEMO ACCOUNTS

The application automatically seeds verified test accounts with Indian phone numbers and addresses during startup:

Portal	Login URL	Email	Password	Role / Details
SUDO Admin	`/login/admin`	`admin@rims.local`	`admin123`	Root Administrator (Inspect all users & delist items)
Seller 1 (Apex Cine Labs)	`/login/seller`	`seller@apexcine.com`	`seller123`	Apex Cine Studios & Media Labs, Mumbai
Seller 2 (TerraForge Fleet)	`/login/seller`	`machinery@terraforge.com`	`heavy123`	TerraForge Heavy Machinery Corp, Bengaluru
Customer / Buyer	`/login/buyer`	`buyer@customer.com`	`buyer123`	Alexander Wright, Film & Construction Logistics

---

🗄️ DATABASE CONFIGURATION & PERSISTENCE

RIMS uses a **persistent file-backed H2 SQL database** (`./data/rimsdb`), requiring zero external database installations for local execution.

- **Database File:** `./data/rimsdb.mv.db` (persisted on disk across server restarts).
- **Web H2 SQL Console:** `http://localhost:8080/h2-console`
  - **Driver Class:** `org.h2.Driver`
  - **JDBC URL:** `jdbc:h2:file:./data/rimsdb`
  - **Username:** `SA`
  - **Password:** *(leave blank)*

Switching to PostgreSQL or MySQL

To switch to PostgreSQL in production, update `src/main/resources/application.properties`:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/rimsdb
spring.datasource.username=postgres
spring.datasource.password=your_secure_password
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
```

🚀 INSTALLATION & EXECUTION GUIDE

Prerequisites

- **Java 21** or later (`java -version`)
- **Maven 3.9+** (`mvn -v`)

1. Build and Run via Maven

```
# Clone or navigate to the project directory
cd rims-app

# Run directly with Spring Boot plugin
mvn spring-boot:run
```

2. Build and Run Standalone Executable JAR

```
# Package into executable JAR
mvn clean package -DskipTests

# Launch application
java -jar target/rental-item-management-1.0.0.jar
```

The application will initialize Tomcat and bind to port `8080`:

- **Public Landing Page:** http://localhost:8080
- **Equipment Catalog:** http://localhost:8080/items
- **Customer Portal:** http://localhost:8080/login/buyer
- **Seller Hub:** http://localhost:8080/login/seller
- **Admin Terminal:** http://localhost:8080/login/admin
- **SQL H2 Console:** http://localhost:8080/h2-console